



Film & Kino Agent

Sitzung 05 – Evaluation von LLM Anwendungen

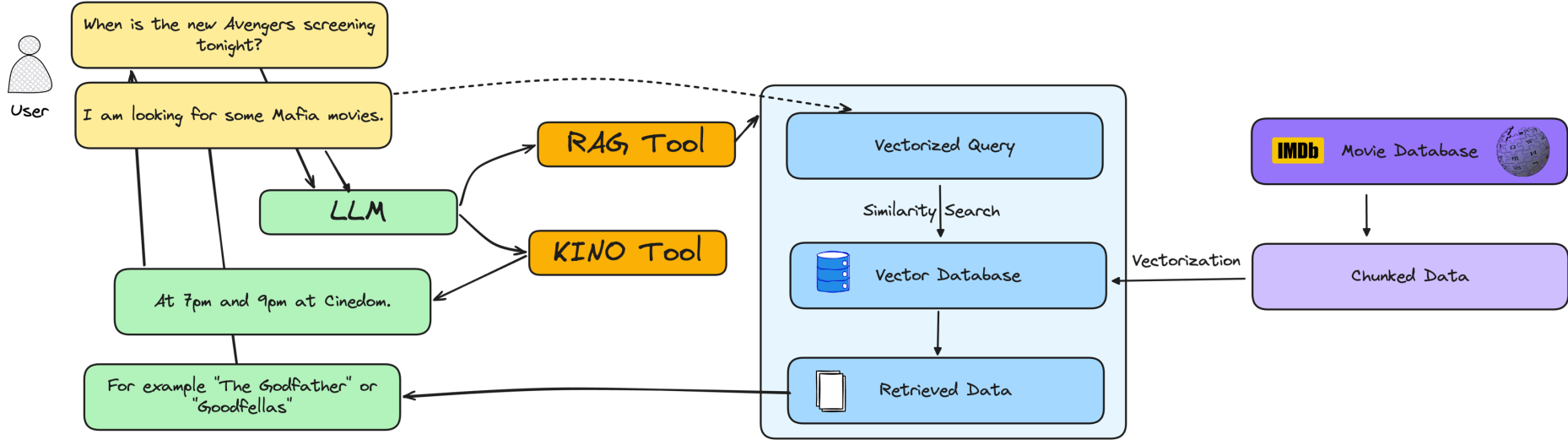
Prof. Dr. Richard Sieg
SoSe 2026

DIS22a

Termine

Datum	Inhalt
15.4	Kick-Off, Projektvorstellung, Setup
22.4	UI bauen, LLM-API-Anbindung, Prompt Engineering
29.4	Embeddings & Vektordatenbanken
6.5	Filmdatenbank vektorisieren
13.5	<i>Home Office</i>
20.5	RAG: Theorie + Implementation (Projektwoche)
27.5	<i>Home Office</i>
3.6	Tool Use & Agenten
10.6	Agenten-Implementation + Theorie: Evaluation
17.6	Gegenseitiger Nutzertest + Feinschliff
24.6	Feinschliff, Bugfixes
1.7	Abschlusspräsentationen

Architektur



Warum Evaluation?

Der Agent läuft. Aber: läuft er gut?

- "Sieht gut aus" ist keine Metrik
- Eine Prompt-Änderung verbessert Fall A, verschlechtert Fall B
- Ohne Messung merkt man Regressionen erst beim Nutzer

Das Kernproblem: LLM-Ausgaben sind nicht-deterministisch und schwer automatisch zu prüfen.

- Klassischer Code: `assert sum(2, 2) == 4`
- LLM: "Empfehl mir Mafia-Filme" hat keine eine richtige Antwort

Zwei Welten der Evaluation

Benchmark - Evals

(das Modell allgemein bewerten)

- Multiple-Choice (z.B. MMLU): Wissensabfrage, Accuracy
- Verifier: prüfbare Aufgaben (Mathe, Code)
- Leaderboards: Modelle gegeneinander (z.B. Chatbot Arena)
- → relevant bei der Modellauswahl, nicht bei der Produktqualität

Produkt - Evals

(konkrete Anwendung bewerten)

- Findet RAG die richtigen Filme?
- Ruft der Agent das richtige Tool auf?
- Halluziniert der Bot?

Was kann man messen?

Was	Wie	Beispiel
Retrieval-Qualität (RAG)	Wurden die richtigen Chunks gefunden?	8 von 10 Testfragen finden relevante Filme
Antwortqualität	Korrekt, vollständig, relevant?	Pass/Fail gegen Kriterien
Halluzinationsrate	Steht alles im Kontext?	Judge prüft Antwort gegen Kontext
Tool-Korrektheit	Richtiges Tool, richtige Parameter?	erwarteter vs. tatsächlicher Aufruf
Latenz & Kosten	Wie schnell, wie teuer?	Tokens/Anfrage, Sekunden/Anfrage
Nutzer*innen Zufriedenheit	Finden Nutzer es hilfreich?	Feedback-Buttons, Umfragen

Retrieval-Metriken

Wenn ein Testset bekannte „richtige“ Dokumente hat, lässt sich Retrieval exakt messen.

Precision@k: Anteil relevanter Treffer unter den Top-k.

- Precision@5 = relevante Treffer in Top 5 / 5

Recall@k: Anteil aller relevanten Dokumente, der in den Top-k auftaucht.

- Recall@5 = relevante Treffer in Top 5 / alle relevanten Dokumente

MRR (Mean Reciprocal Rank): Wie weit oben steht der erste relevante Treffer?

- $RR = 1 / \text{Rang des ersten relevanten Treffers}$
- MRR = Durchschnitt über alle Anfragen

Retrieval-Metriken: Beispiel

Query: "einsamer Astronaut im Weltraum"

3 relevante Filme in der DB (Interstellar, Gravity, The Martian).

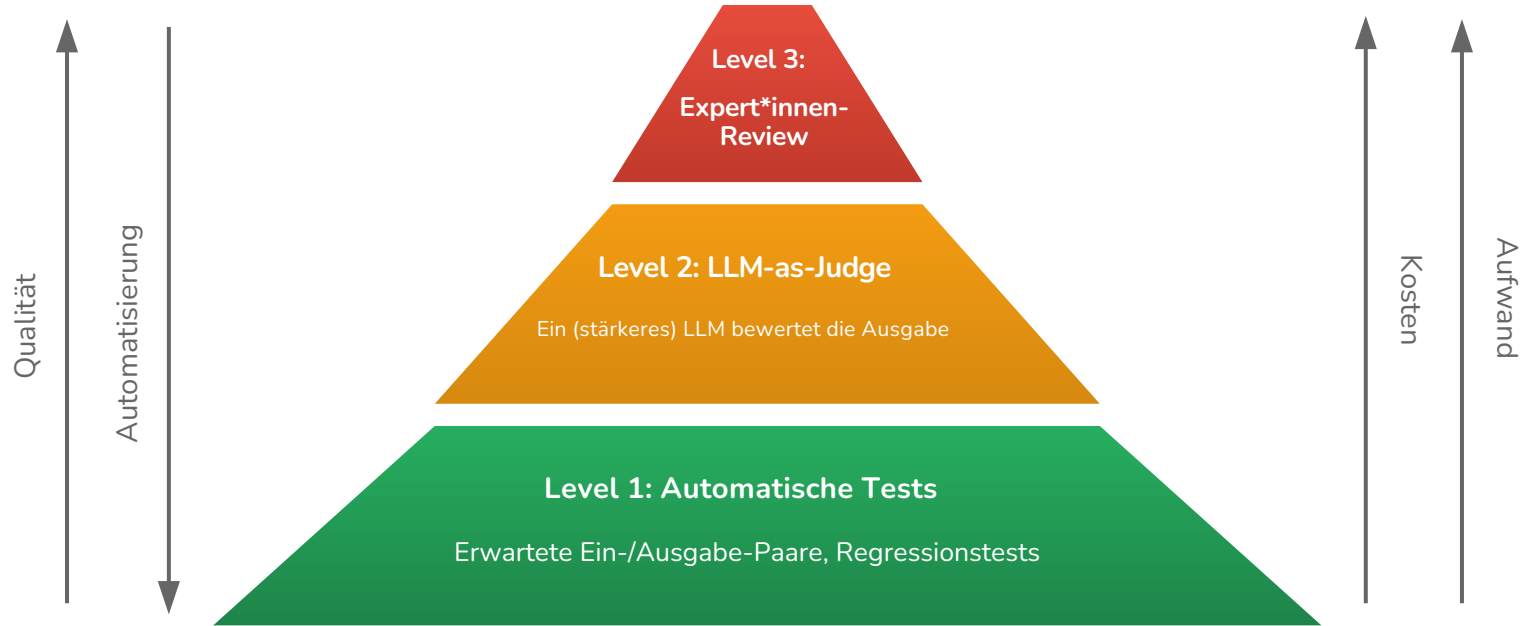
Rang	Film	relevant?
1	Gravity	✓
2	Armageddon	✗
3	Interstellar	✓
4	Apollo 13	✗
5	The Martian	✓

- **Precision@5** = $3/5 = 0.60$
- **Recall@5** = $3/3 = 1.00$
- **MRR** = $1/1 = 1.00$ (erster Treffer auf Rang 1)

Die Evaluationspyramide



Regel Nummer 1
Immer die LLM Aufrufe anschauen



Übersicht der drei Ebenen

	Automatischer Test	LLM-as-Judge	Expert*innen-Review
Was?	z.B. Tool gibt korrekte Treffer zurück	z.B. Antwort basiert auf Kontext	Gesamtverhalten des Chatbots
Testdaten	z.B. 20 vordefinierte Suchanfragen	z.B. 50 Frage-Antwort-Paare	z.B. 10 repräsentative Fälle
Ergebnis	z.B. min. 80% richtig	z.B. Halluzinationsrate < 10%	z.B. Rating $\geq 4/5$
Aufwand	gering (einmalig erstellen)	mittel (Prompt + Auswertung)	hoch (Personalzeit)
Kadenz	bei jedem Commit	in festem Rhythmus (zB vor jedem Release)	punktuell, nach größeren Änderungen

Level 1: Automatische Tests

Deterministische Checks ohne LLM. Schnell (Millisekunden), kostenlos, perfekt reproduzierbar.

Was sich gut so testen lässt:

- Tool-Korrektheit: wurde `search_movies_rag` mit `genre="Crime"` aufgerufen?
- Retrieval: ist der erwartete Film unter den Top-k?
- Format: ist die Ausgabe valides JSON, enthält sie ein Rating?
- Generisch: enthält die Antwort keine internen IDs?
- Regression: liefert ein bekannter Input weiter das bekannte Ergebnis?

Pydantic Evals

```
from pydantic_evals import Case, Dataset
from pydantic_evals.evaluators import IsInstance, Contains

dataset = Dataset(
    cases=[
        Case(name="mafia_query",
             inputs="Empfehl mir Mafia-Filme",
             expected_output="The Godfather"),
    ],
    evaluators=[
        IsInstance(type_name="str"),
        Contains(value="The Godfather"),
    ],
)
report = dataset.evaluate_sync(my_agent)
report.print()
```

Level 1: Best Practices

- Pass-Rate ist eine Produktentscheidung: anders als bei klassischen Tests braucht man meist keine 100%. Welche Fehlerquote ist tolerierbar?
- Billig vor teuer: erst Typ- und Format-Checks, dann teure Checks.
- Testfälle synthetisch erzeugen: ein LLM kann z.B. „50 verschiedene Film-Anfragen“ generieren, die alle Szenarien abdecken. Hier helfen auch Persona Bibliotheken.
- Tool-Calls prüfen, nicht nur Endausgabe: bei Agenten ist der Pfad (welches Tool, welche Argumente) oft wichtiger als der Text.
- Jeder Bug wird ein Testfall: so kehren Fehler nicht zurück (*Regression*).
- Schnell halten: Level-1-Tests sollen in Sekunden durchlaufen, sonst werden sie nicht ausgeführt.

Level 2: LLM-as-Judge

Ein (stärkeres) LLM bewertet die Ausgabe eines anderen. Nützlich für alles, was sich nicht deterministisch prüfen lässt: Relevanz, Ton, Vollständigkeit, Groundedness.

```
from pydantic_evals.evaluators import LLMJudge

LLMJudge(
    rubric="Die Antwort nutzt nur Filme aus dem "
          "bereitgestellten Kontext, erfindet keine Titel",
    include_input=True,
    assertion={"evaluation_name": "grounded",
              "include_reason": True},
)
```

Typische Einsätze für den Film-Agenten:

- Groundedness: Steht jede Filmempfehlung im Retrieval-Kontext?
- Relevanz: Passt die Antwort zur Frage?
- Ton: Antwortet der Bot in der Sprache des Nutzers?

Level 2: Drei Bewertungs-Ansätze

Direct Scoring (eine Antwort einzeln bewerten)

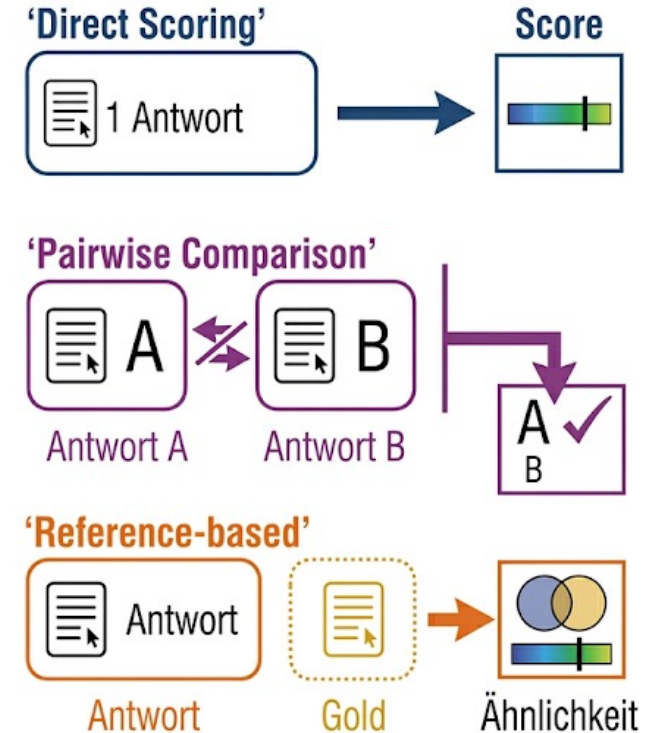
- gut für objektive Kriterien: Faithfulness, Toxicity
- „Ist diese Antwort faithful zum Kontext? Ja/Nein“
- braucht keine Vergleichsantwort

Pairwise Comparison (zwei Antworten vergleichen)

- gut für subjektive Kriterien: Ton, Kohärenz
- stabiler als Direct Scoring bei subjektiven Fragen
- „Welche Antwort ist besser, A oder B?“

Reference-based (gegen eine Musterlösung)

- vergleicht Antwort mit einer Gold-Referenz
- braucht annotierte Referenzen



Level 2: Best Practices (1 / 2)

- Binär statt Likert
Pass/Fail, nicht 1–5. Niemand weiß, was eine 3 von einer 4 unterscheidet, und selbst Menschen labeln Skalen inkonsistent.
- Eine Eigenschaft pro Judge
Kein „God Evaluator“, der 5–10 Dimensionen auf einmal prüft. Je ein Judge für Groundedness, Relevanz, Ton. Dann sieht man, welche Dimension das Problem ist.
- Begründung verlangen
Der Judge soll warum sagen. Die Kritik ist oft wertvoller als das Urteil.
- Generische Metriken vermeiden
„Helpfulness-Score“, „Coherence-Score“ aus dem Framework klingen gut, sind aber zu generisch um echte Fehler zu finden. Besser anwendungsspezifisch: „Falscher Film empfohlen“, „Genre-Filter ignoriert“.

Level 2: Best Practices (2 / 2)

Den Judge wie einen Klassifikator behandeln und validieren:

- Ein Judge ist eine Black Box, die Vorhersagen macht. Wie sehr vertraut man ihr?
- Domänenexpert*in labelt z.B. 50–100 Fälle manuell (Pass/Fail + Begründung)
- Daten in Dev/Test splitten (z.B. 75/25): Judge-Prompt am Dev-Set optimieren, am Test-Set prüfen, dass er nicht overfittet
- Mit Precision & Recall berichten, nicht Accuracy. Tritt ein Fehler nur in 5% der Fälle auf, versteckt Accuracy die wahre Leistung. Recall auf der „Fail“-Klasse ist meist am wichtigsten (Defekte fangen).
- Cohen's Kappa misst Übereinstimmung mit Labels von Menschen oder anderem Modell: 0.4–0.6 = substantiell, > 0.7 = exzellent

Level 2: Bekannte Biases

- Position Bias
Bei Pairwise wird oft die erste Antwort bevorzugt → Reihenfolge tauschen, zweimal bewerten
- Length Bias
Längere Antworten werden oft besser bewertet
- Self-Preference
Ein Modell bevorzugt den eigenen Stil

RAGAS: Fertige RAG-Metriken

Ragas bündelt LLM-as-Judge-Metriken speziell für RAG.

- Faithfulness: bezieht sich die Antwort nur auf den Kontext? (Halluzinations-Check)
- Answer Relevancy: beantwortet sie die Frage?
- Context Precision: Signal-to-Noise im abgerufenen Kontext
- Context Recall: wurden alle relevanten Infos gefunden?

⚠ „Ready-to-use“ Metriken sind ein guter Start, aber kein Ersatz für eigene, anwendungsspezifische Kriterien.

```
from ragas import evaluate
from ragas.llms import LangchainLLMWrapper

evaluator_llm = LangchainLLMWrapper(llm)
from ragas.metrics import LLMContextRecall, Faithfulness, FactualCorrectness

result = evaluate(dataset=evaluation_dataset, metrics=[LLMContextRecall(),
Faithfulness(), FactualCorrectness()], llm=evaluator_llm)
```

Level 3: Expert*innen-Review

Wann nötig:

- subtile Qualität, die Metriken nicht erfassen (zB Empfehlungs Geschmack)
- Allgemeine Einschätzung des Systems
- zur Validierung der automatischen Judges (siehe Level 2)
- bei kritischen oder mehrdeutigen Fällen

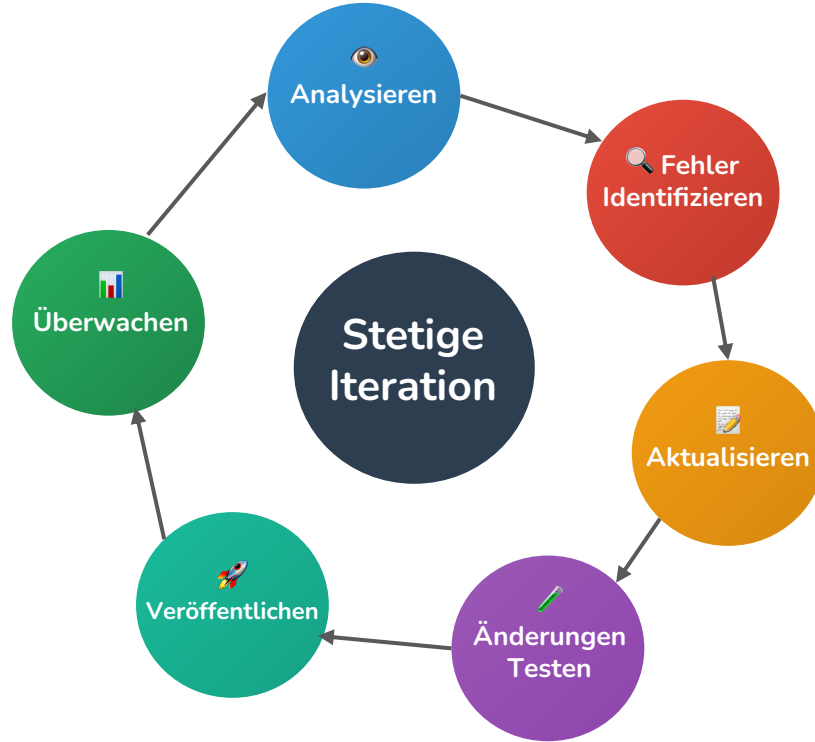
Best Practices:

- Eine Hauptexpertin als Standard („benevolent dictator“). Verschiedene Leute haben verschiedene Meinungen; entscheidend ist, dass die Hauptexpert*in zufrieden ist.
- repräsentative, diverse Fälle (verschiedene Features, Szenarien, Personas)
- Pass/Fail mit Begründung, gleiche Disziplin wie beim Judge
- Ergebnisse fließen zurück in Testset und Judge-Rubriken

Häufige Fehler (Anti-Patterns)

- Generische Metriken
Dashboard voller „Helpfulness/Hallucination“-Scores, die nichts diagnostizieren.
→ Erst Traces lesen, Fehler kategorisieren (Error Analysis), dann passende Metriken
- Unverifizierte Judges
Dem LLM-Judge blind vertrauen.
→ Wie einen Klassifikator behandeln, gegen menschliche Labels prüfen.
- Schlechtes Testset-Design
„Gib mir 50 Test-Queries“ erzeugt generische, unrepräsentative Daten.
→ In echten Logs/Traces verankern, gezielt Edge Cases einstreuen.
- Schlechte Daten & Labels
Labeln an andere delegieren.
→ Domänenexpert*innen labeln lassen, skeptisch bleiben.
- Zu viel automatisieren
Das LLM kann das Plumbing schreiben, aber nicht für dich die Daten anschauen.

Der Produktzyklus



Wichtigste Erkenntnisse

- ✓ Aufrufe analysieren
- ✓ Einfach anfangen
- ✓ Aufgabenspezifische Evals
- ✓ Prüfer validieren
- ✓ Kontinuierlich iterieren

Aufgabe: Gegenseitiger Chatbot-Test

- Ziel: Ihr baut ein Testset und testet damit die Agenten der anderen Gruppen. Das ist Level 3 der Pyramide, angewandt aufeinander.
- Schritt 1: Testset bauen (als Tabelle)
Mindestens 15 Testfälle, die RAG und Tool Use abdecken. Mindestens Spalten:

Anfrage	Typ	Erwartete Ausgabe	Generierte Ausgabe	Kommentar / Pass-Fail
"Empfehl Mafia-Dramen aus den 2000ern"	RAG + Filter	The Departed, ...	(leer)	(leer)
"Was läuft gerade?"	Tool (Charts)	aktuelle TMDB-Liste	(leer)	(leer)
"Trailer zu Avatar"	nicht unterstützt	ehrliche Absage	(leer)	(leer)

- Testet das schonmal auf eurem Chatbot
- Nächste Woche testet ihr das dann auf jedem anderen Chatbot

Aufgabe: Hinweise

- In echten Anfragen verankern
Keine generischen Lehrbuch-Queries. Überlegt, was Nutzer wirklich fragen würden, und streut Edge Cases ein.
- Erwartete Ausgabe nicht zu eng fassen
Bei RAG gibt es selten die eine Antwort. Beschreibt, was eine gute Antwort enthalten muss (z.B. „mindestens 2 Crime-Filme aus 2000–2009“).
- Binär bewerten
Pass/Fail mit Begründung, keine 1–5-Skala. „Fail, weil Film X halluziniert wurde“ ist wertvoller als ein nur Fail.
- Tool-Pfad mitdenken
Notiert auch, welches Tool aufgerufen werden sollte, nicht nur den finalen Text.
- Diverse Dimensionen
verschiedene Features (RAG, Charts, Filter), Szenarien (Treffer, kein Treffer, mehrdeutig) und Personas (Genre-Fan, Gelegenheitsgucker).

Quellen

Hamel Husain hamel.dev/blog/posts/

- Your AI Product Needs Evals · Creating a LLM-as-a-Judge That Drives Business Results · LLM Evals: Everything You Need to Know (FAQ) · The Revenge of the Data Scientist

Eugene Yan, eugeneyan.com/writing/

- Evaluating LLM-Evaluators (LLM-as-Judge) · Product Evals in Three Simple Steps

Weitere:

- Sebastian Raschka: Understanding the 4 Main Approaches to LLM Evaluation, magazine.sebastianraschka.com
- Shankar et al. (2024): Who Validates the Validators? (Criteria Drift), [arXiv:2404.12272](https://arxiv.org/abs/2404.12272)
- Pydantic Evals Docs, ai.pydantic.dev/evals/ · Ragas Docs, docs.ragas.io